

# Operating Systems

Rowan University

# Overview

1. Paging Algorithms Review
2. Design Issues
3. Implementation Issues
4. Project Part 1

# Page Table

Figure 3-10

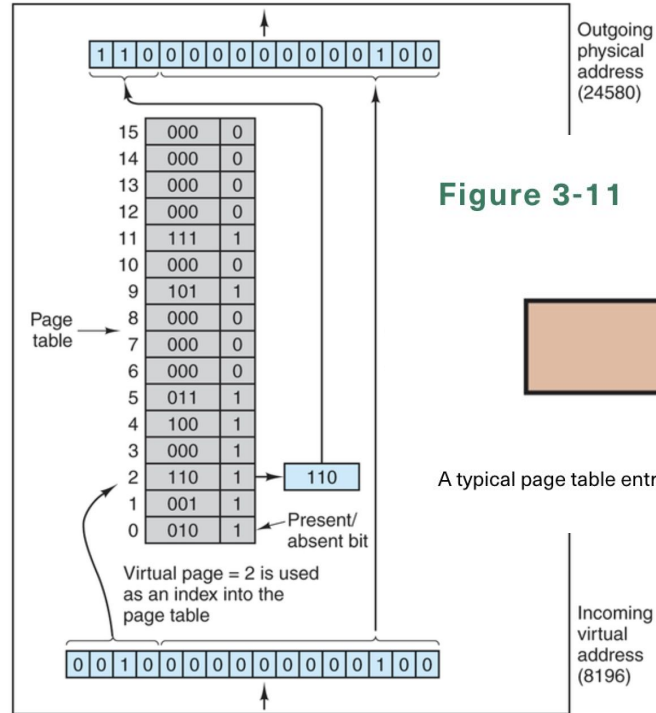
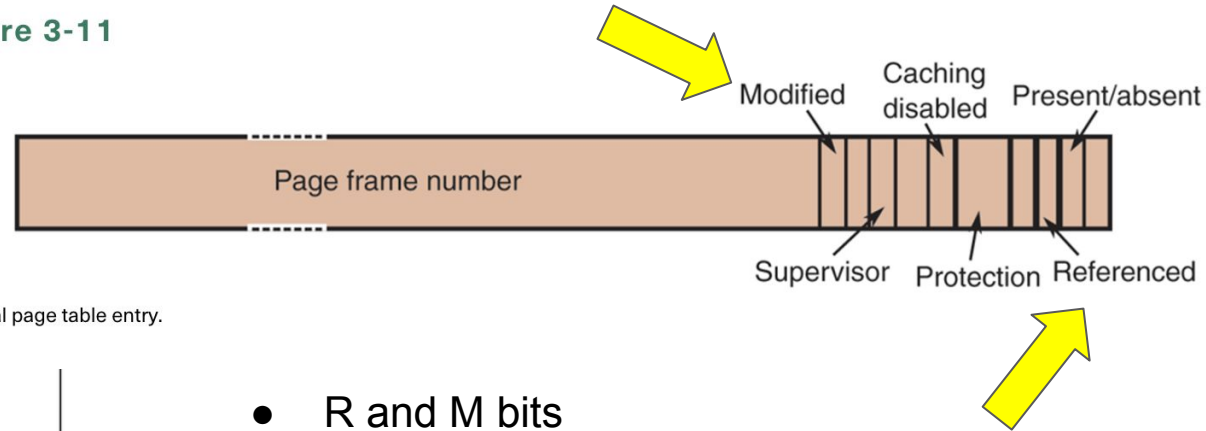


Figure 3-11



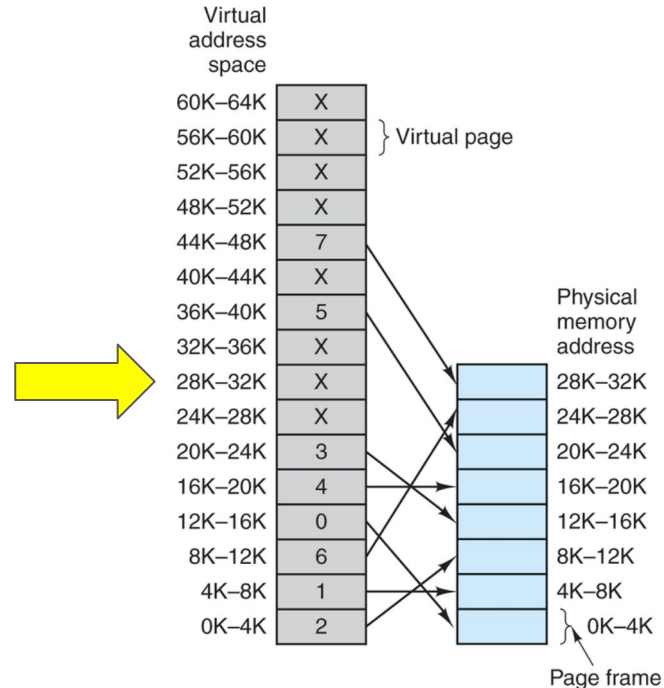
- R and M bits
- Has this been referenced?
- Has this been modified?

# Page Fault

**Page Fault** - The MMU sees an unmapped page and causes the CPU to *trap* to the operating system

1. The OS picks a page *frame* and writes it back to disk
2. Then, fetches the page that was just referenced into the page frame, updates the map
3. Restarts the *trapped* instruction

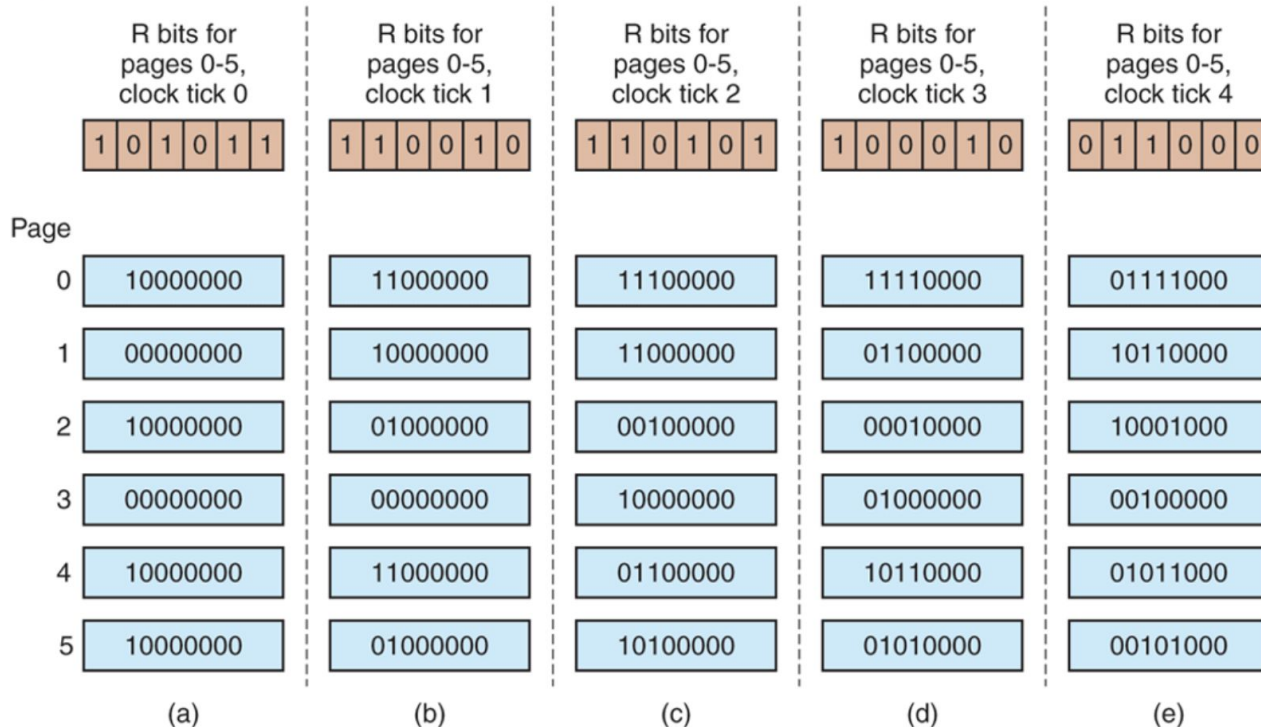
Figure 3-9



The relation between virtual addresses and physical memory addresses is given by the **page table**. Every page begins on a multiple of 4096 and ends 4095 addresses higher, so 4K-8K really means 4096-8191 and 4K-8K means 8192-12287.

# Least Recently Used (LRU) Algorithm

Figure 3-17

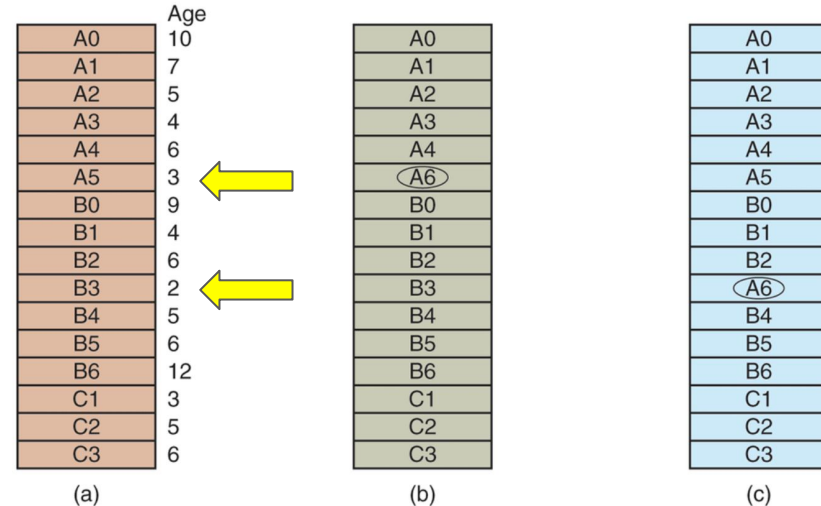


The aging algorithm simulates LRU in software. Shown are six pages for five clock ticks. The five clock ticks are represented by (a) to (e).

# Local versus Global Allocation Policies

- How should memory be allocated among processes?
- Locally LRU, remove A5
- Global LRU, remove B3
- Considerations:
  - How many page frames per process?
  - What is the working set size of a process?
  - Does the working set size change?
- Goals:
  - Avoid thrashing
  - Avoid wasted memory

Figure 3-22

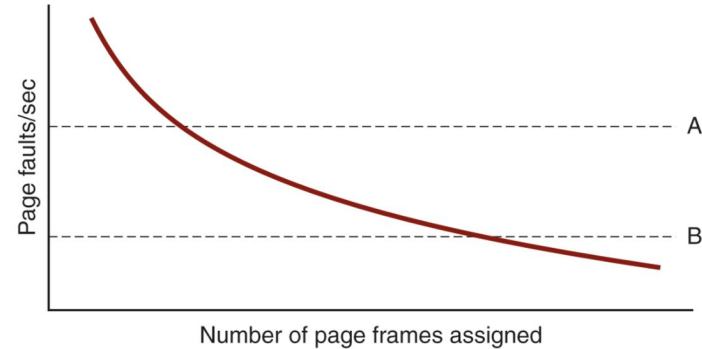


Local versus global page replacement. (a) Original configuration. (b) Local page replacement. (c) Global page replacement.

# Page Fault Frequency (PFF) Algorithm

- **Page Fault Frequency**
  - Monitor page faults for a process
- **Thresholds**
  - Above A faults/second, needs *more* memory
  - Below B faults/second, needs *less* memory
- If all processes are at the A threshold, then the system is in a *thrashing* state

Figure 3-23

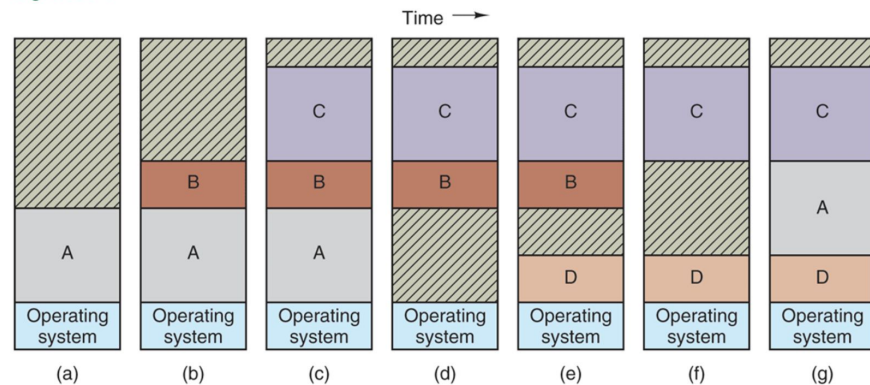


Page fault rate as a function of the number of page frames assigned.

# Load Control

- Process memory requirements exceed physical memory
- **Load Control**
  - Out of Memory (OOM) Killer
  - *Swap* an entire process to storage
- **Clean up:**
  - Compaction
  - Compression
  - Deduplication
- **Same page merging**
  - If two processes are reading, they can both reference the page
  - Copy on write occurs once a process modifies the page

Figure 3-4



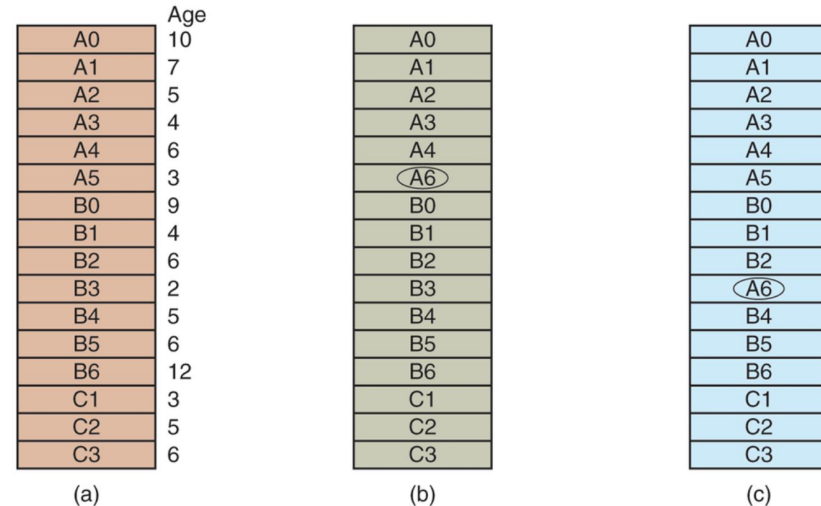
Memory allocation changes as processes come into memory and leave it. The shaded regions are unused memory.



# Cleaning Policy

- Paging systems run as a background process
  - Paging *daemon*, mostly asleep
- Frees page frames
  - Writes it back to disk
  - Clears the memory location
- Clears the memory to avoid writing to disk while a process is running
  - Writes are expensive
- Performance improves when there are clean page frames that can be used

Figure 3-22

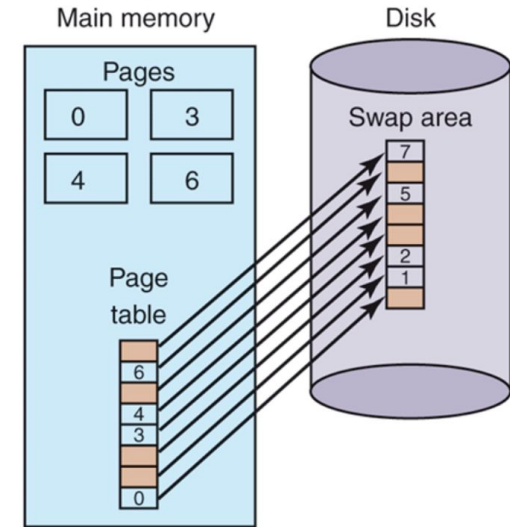


Local versus global page replacement. (a) Original configuration. (b) Local page replacement. (c) Global page replacement.

# Page Sizes

- How big should a page frame be?
- Operating system decides
  - Hardware may be built for 4KB frames
  - OS can pair frames to make larger ones
- Internal Fragmentation
  - Page is not full
  - Last page 50% full
- Page Table Size
  - Smaller pages
    - Larger page table
    - Use up space in TLB
  - Lives in memory
  - Storage reads/writes are 1 page at a time
- Huge Pages

Figure 3-28



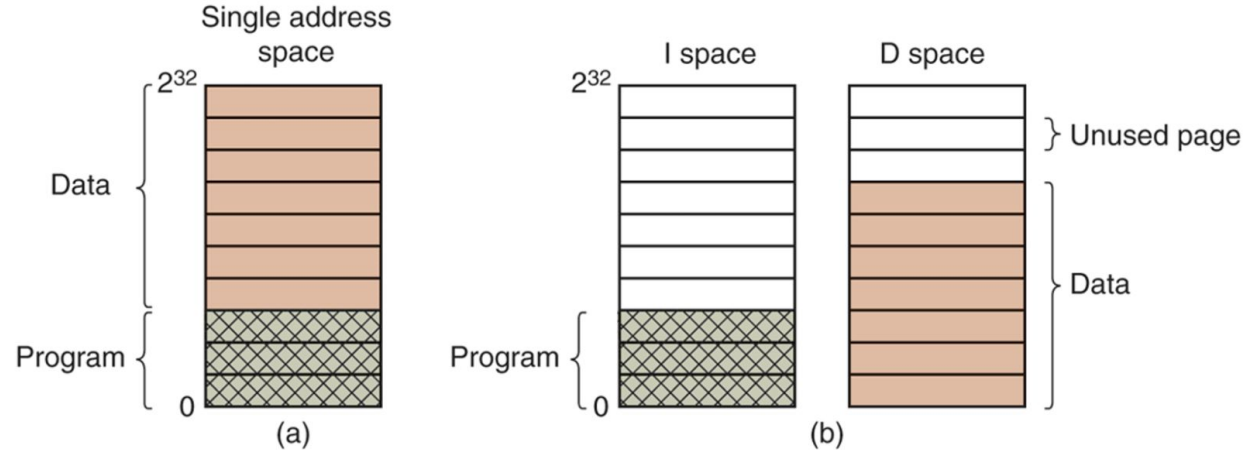
(a)

(a) Paging to a static swap area. (b) Backing up pages dynamically.

# Separate Instructions and Data Spaces

- Separate Page Tables
  - Instruction space
  - Data space

Figure 3-24

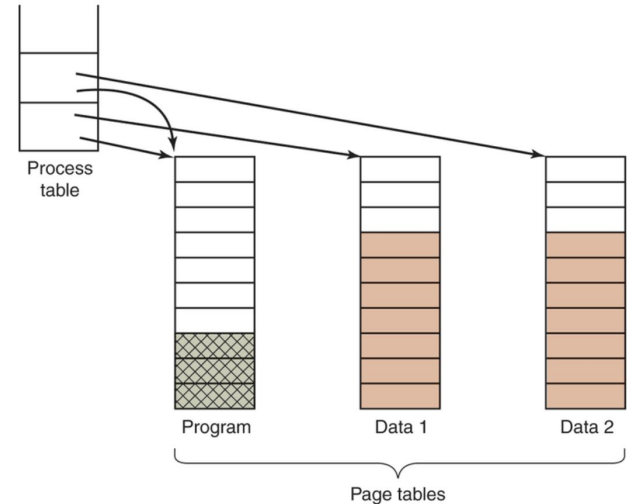


(a) One address space. (b) Separate I and D spaces.

# Shared Pages

- Processes share the instruction space
- Process table includes 2 pointers to memory locations
  - I-space pointer
  - D-space pointer
- Many users are running the same program
- Issue when removing a process sharing I-space
  - Removing the program will then generate page faults once context switches to another program
- Sharing data is possible
  - Recall “copy on write”

Figure 3-25

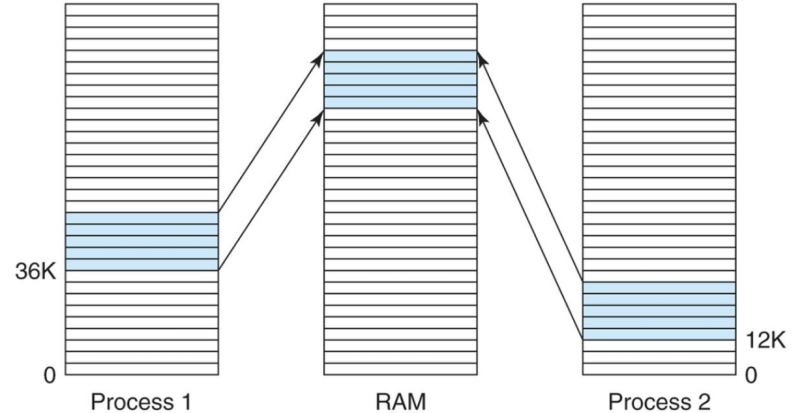


Two processes sharing the same program sharing its page tables.

# Shared Libraries

- Processes will use the same libraries
  - No need to copy these multiple times
  - Considered undefined external functions
- The Linker will take care of linking to shared libraries
  - Includes only what is needed
  - Includes a small stub routine that binds to the function at runtime
- Shared libraries loaded by one process are available to other processes

Figure 3-26



A shared library being used by two processes.

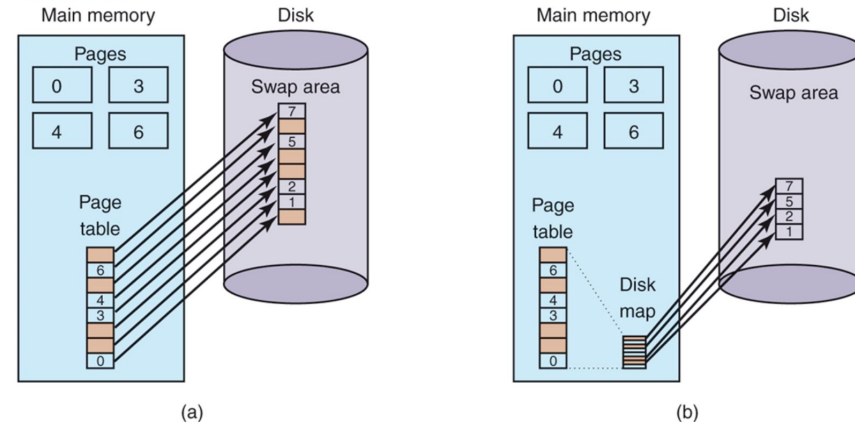
# Implementation Issues

- Process creation time
  - Needs to allocate space in memory for the process
  - Page table initialization
- Process execution time
  - Reset MMU, clear TLB,
  - Registers updated
- Page fault time
  - Locate the page to load
  - Find the page to evict
- Process termination time
  - Release memory, swap space used

# Backing Store

- Swap space
  - A partition on the disk that does not have a traditional file system on it
  - Lower overhead to accessing this space
  - A list of free chunks of storage
- Processes can swap in and out of memory

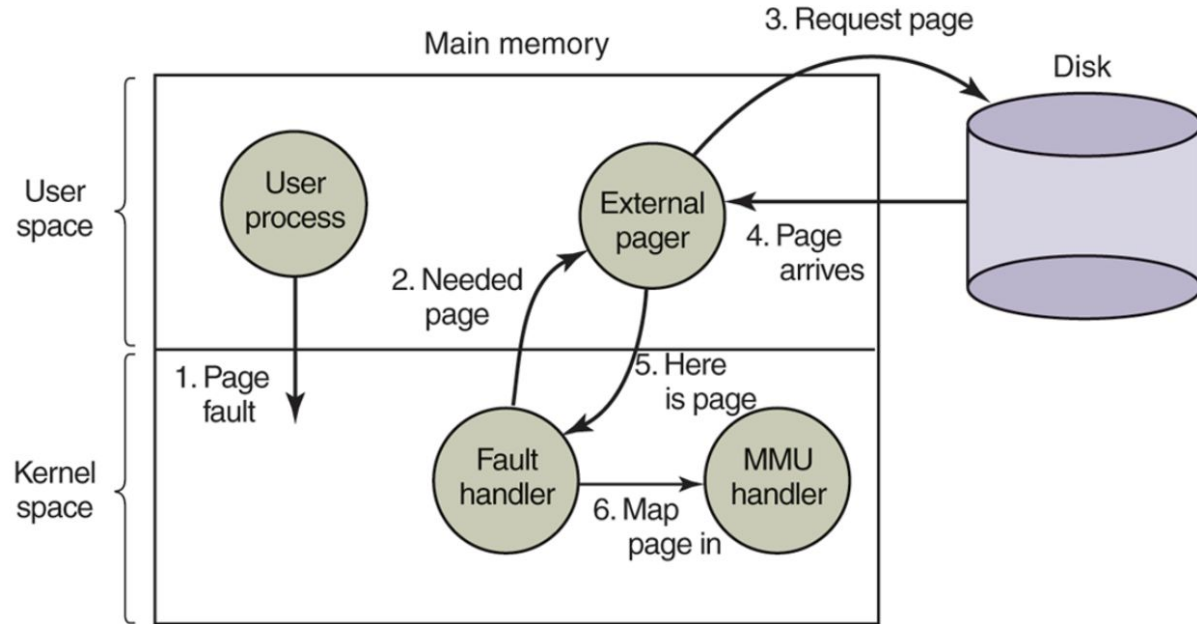
Figure 3-28



(a) Paging to a static swap area. (b) Backing up pages dynamically.

# Page Fault Handling

Figure 3-29



Page fault handling with an external pager.



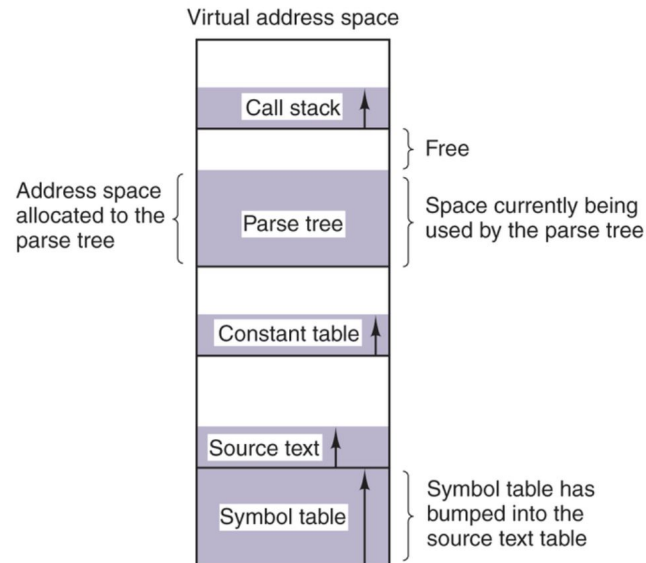
# Page Fault Handling

- **Hardware Trap and Register Save:**
  - Hardware traps to the kernel, saving the program counter on the stack.
  - Special CPU registers store essential information about the current instruction.
- **Interrupt Service Routine (ISR) Initialization:**
  - An assembly-code ISR starts to save registers and crucial data to prevent OS interference.
  - The ISR then invokes the page fault handler.
- **Virtual Page Identification:**
  - The OS determines the needed virtual page, often extracting this information from hardware registers.
  - If not available, the OS retrieves the program counter, fetches the instruction, and parses it.
- **Validation and Protection Check:**
  - Verify if the virtual address causing the fault is valid and has consistent protection.
  - Signal or terminate the process if the address is invalid; otherwise, proceed to the next step.
- **Page Frame Management:**
  - Check for free page frames; if none, run the page replacement algorithm to select a victim.
  - If the chosen page frame is dirty, schedule transfer to nonvolatile storage and initiate a context switch.
- **Disk or SSD Operation:**
  - Mark the frame as busy to prevent other uses.
  - Once the frame is clean, look up the disk address for the needed page and schedule the loading operation.
- **Process Suspension and Resumption:**
  - Suspend the faulting process during the page loading operation.
  - Resume another user process if available.
- **Page Arrival and Update:**
  - Update page tables upon disk or SSD interrupt indicating page arrival.
  - Mark the frame as in the normal state.
- **Instruction Backup and Program Counter Reset:**
  - Backup the faulting instruction to its initial state.
  - Reset the program counter to point to that instruction.
- **Process Scheduling and ISR Conclusion:**
  - Schedule the faulting process for execution.
  - Return to the assembly-language routine, reload registers and state information, and resume execution in user space.

# Segmentation

- 1D address space
- Different segments of the address space grow and shrink
- What happens when one section grows too large?
- Free space represents allocated but unused, impacts utilization

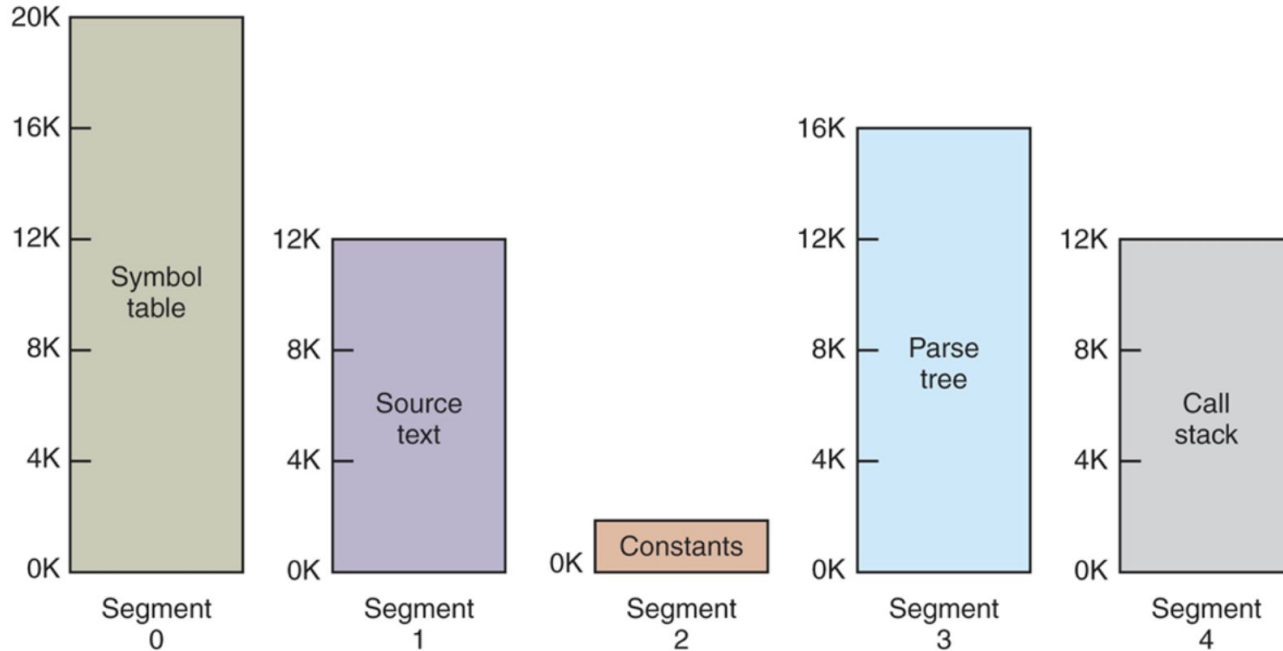
Figure 3-30



In a one-dimensional address space with growing tables, one table may bump into another.

# Segmentation

Figure 3-31



A segmented memory allows each table to grow or shrink independently of the other tables.